TourGuide

Documentation fonctionnelle et technique

Sommaire

1. Présentation du projet
   1.1 Objectifs du projet
   1.2 Hors du champ d'application
   1.3 Mesures du projet
2. Spécifications fonctionnelles
3. Spécifications techniques
   3.1 Schémas de conception technique
   3.2 Glossaire
   3.3 Solutions techniques
   3.4 Autres solutions non retenues

1. Présentation du projet

TourGuide est une application backend Spring Boot qui facilite la planification de voyages en s’appuyant sur la géolocalisation. Elle permet de suivre la position d’un utilisateur, de lui recommander des attractions touristiques à proximité, de calculer des points de récompense en fonction de ses visites, et de proposer des offres de voyage personnalisées via des services externes (GPS, récompenses, tarification). Le public cible est large : utilisateurs grand public (application mobile ou web), mais aussi l’entreprise TourGuide elle-même, qui doit garantir une expérience fluide malgré une forte croissance. Sur le plan commercial, l’intérêt est double : améliorer l’engagement (recommandations pertinentes + gamification via récompenses) et soutenir la montée en charge afin d’éviter des lenteurs qui dégradent l’expérience et freinent la croissance.

1.1 Objectifs du projet

L’objectif du projet est d’améliorer la fiabilité et les performances de l’application face à une hausse rapide du nombre d’utilisateurs, avec une cible pouvant atteindre 100 000 utilisateurs. Le projet résout des problèmes de concurrence (tests instables, comportements intermittents) et optimise les traitements coûteux (géolocalisation et calcul des récompenses) afin de respecter des seuils de performance imposés tout en corrigeant la fonctionnalité de recommandations des 5 attractions les plus proches.

1.2 Hors du champ d’application

Ce projet ne couvre pas une refonte fonctionnelle globale de l’application ni l’ajout de nouvelles fonctionnalités métier majeures au-delà de la correction demandée sur les attractions proches. Les

évolutions fonctionnelles introduites restent ciblées (endpoints utilitaires de support/démo). Il n’inclut pas non plus la mise en place d’une base de données persistante (les utilisateurs restent gérés en mémoire dans le cadre du projet), ni un travail sur l’authentification/sécurité, ni une refonte d’interface front-end. Enfin, il ne vise pas une transformation architecturale lourde (microservices, déploiement cloud, scalabilité horizontale), l’objectif étant de rester dans une amélioration ciblée et pragmatique des performances et de la stabilité.

1.3 Mesures du projet

Le succès du projet se mesure par :

- la stabilité des tests (absence d’échecs intermittents liés à la concurrence),
- la conformité fonctionnelle de la recommandation des “5 attractions les plus proches”,
- et surtout l’atteinte des objectifs de performance vérifiés par les tests de charge : obtenir 100 000 localisations utilisateurs en moins de 15 minutes et calculer les récompenses pour 100 000 utilisateurs en moins de 20 minutes, avec mise à jour des résultats dans les mesures de performance demandées.

2. Spécifications fonctionnelles

L’application TourGuide propose un ensemble de fonctionnalités centrées sur la géolocalisation des utilisateurs et l’aide à la planification de voyages. Elle permet tout d’abord d’identifier un utilisateur et de récupérer sa dernière position connue, ou, si nécessaire, de déclencher un suivi de sa position via un service de géolocalisation. À partir de cette position, l’application est capable de recommander automatiquement les cinq attractions touristiques les plus proches de l’utilisateur, sans contrainte de distance maximale, tout en fournissant pour chacune d’elles des informations utiles telles que la localisation, la distance par rapport à l’utilisateur

et le nombre de points de récompense potentiels.

L’application gère également un système de récompenses : lorsqu’un utilisateur visite une attraction, des points lui sont attribués en fonction des règles définies par un service externe de calcul des récompenses. Ces récompenses sont cumulées et associées à l’utilisateur, sans doublon pour une même attraction. En complément, TourGuide propose des offres de voyage personnalisées en s’appuyant sur les préférences de l’utilisateur (nombre d’adultes, d’enfants, durée du séjour) et sur le total de ses points de récompense, grâce à l’intégration d’un service externe de tarification.

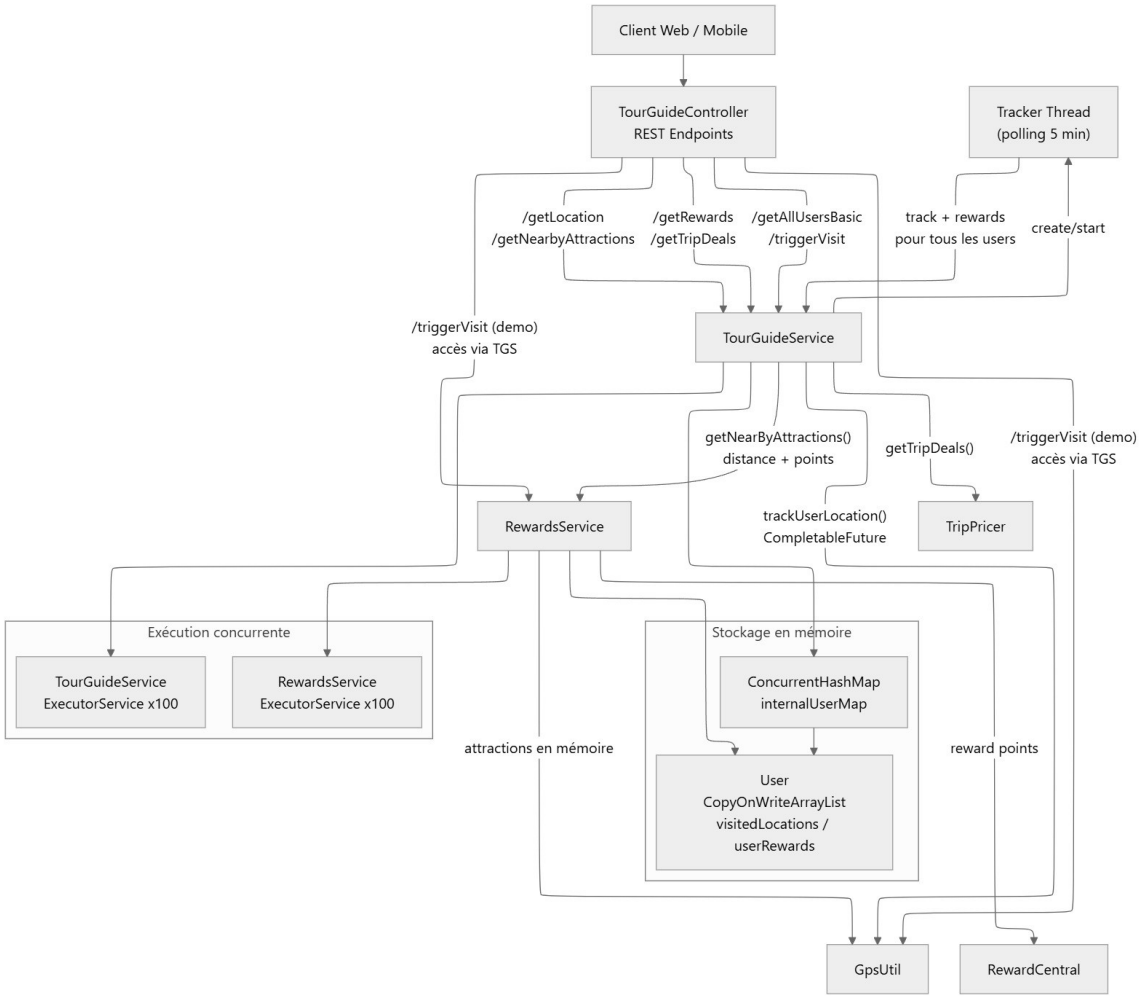
Enfin, l’application expose ces fonctionnalités via des endpoints REST consommables par un front-end ou une application mobile :

- `/getLocation` (position de l’utilisateur),
- `/getNearbyAttractions` (5 attractions les plus proches),
- `/getRewards` (récompenses utilisateur),
- `/getTripDeals` (offres de voyage),
- `/getAllUsersBasic` (liste limitée à 10 utilisateurs pour alimenter l’UI),
- `/triggerVisit` (endpoint de démo/test qui force une visite et déclenche le calcul des récompenses).

Elle inclut également un suivi automatique des utilisateurs via un thread de fond (Tracker), avec mise à jour périodique de la position et recalcul des récompenses.

3. Spécifications techniques

3.1 Schémas de conception technique



L’architecture de l’application TourGuide repose sur une structure classique Spring Boot en couches, renforcée par des mécanismes de concurrence et de traitement asynchrone afin de répondre aux contraintes de performance.

L’application expose des endpoints REST via le contrôleur TourGuideController, qui constitue le point d’entrée principal pour les clients (front-end ou application mobile). Le contrôleur délègue l’essentiel des traitements au service applicatif TourGuideService. Il contient aussi une logique légère de validation/erreur (retour 404 si utilisateur introuvable) et un endpoint utilitaire de démo (`/triggerVisit`) qui force une visite puis déclenche un calcul de récompenses. Pour ce point de démo, le contrôleur passe via TourGuideService pour accéder à GpsUtil et RewardsService, ce qui reste volontairement pragmatique mais un peu moins strict qu’une

orchestration métier encapsulée dans une seule méthode de service.

Le cœur métier est concentré dans TourGuideService. Ce service orchestre les différents traitements : récupération de la position des utilisateurs, calcul des attractions proches, gestion des récompenses et génération des offres de voyage. Il s’appuie sur des appels asynchrones (CompletableFuture) et sur un pool de threads dédié afin de paralléliser les opérations coûteuses, notamment les appels au service de géolocalisation. Cette approche permet d’éviter le blocage du thread principal et d’absorber une charge importante d’utilisateurs simultanés.

Dans l’implémentation actuelle, le service fonctionne avec un mode interne actif (`testMode = true`) qui initialise automatiquement des utilisateurs en mémoire (nombre piloté par `InternalTestHelper`, valeur par défaut : 100 000). Ce point est important pour comprendre le comportement au démarrage et les tests de charge.

Le calcul des récompenses est isolé dans RewardsService, qui encapsule la logique liée à la proximité des attractions et à l’attribution des points. Les attractions sont chargées en mémoire au démarrage de l’application, ce qui limite les appels répétés à des services externes et améliore significativement les performances. Ce service utilise également un pool de threads afin de traiter les calculs de récompenses de manière concurrente.

Un composant spécifique, le Tracker, fonctionne comme un thread d’arrière-plan. Il déclenche périodiquement la mise à jour de la position des utilisateurs et le recalcul de leurs récompenses (intervalle de polling : 5 minutes). Le Tracker coordonne des traitements asynchrones et attend leur complétion avant de passer au cycle suivant, ce qui garantit un contrôle de la charge et évite l’accumulation incontrôlée de tâches.

Les services externes (GpsUtil pour la géolocalisation, RewardCentral pour les points de récompense et TripPricer pour les offres de voyage) sont consommés par les services métier. Dans l’implémentation actuelle, GpsUtil et RewardCentral sont injectés comme beans Spring, tandis que TripPricer est instancié directement dans TourGuideService. L’architecture est pensée pour limiter leur impact sur les performances globales grâce à la parallélisation et à la mise en cache partielle des données.

Enfin, les données utilisateurs sont gérées en mémoire, à l’aide de structures thread-safe (ConcurrentHashMap, CopyOnWriteArrayList) afin de garantir la cohérence des données en environnement concurrent. L’implémentation actuelle ajoute aussi des garde-fous : historique des localisations limité à 30 entrées par utilisateur et synchronisation explicite lors de l’ajout des récompenses pour éviter les doublons. Cette architecture permet à l’application de rester simple et suffisamment robuste pour une montée en charge importante dans le cadre du projet, principalement grâce à l’optimisation intra-process (concurrence et mémoire) plutôt qu’à une scalabilité horizontale distribuée.

3.2 Glossaire

Dans le contexte du projet TourGuide, un utilisateur désigne une personne utilisant l’application pour planifier un voyage. Chaque utilisateur est identifié de manière unique et possède des préférences (durée du séjour, nombre d’adultes et d’enfants), un historique de positions géographiques et un ensemble de récompenses associées à ses visites.

Une localisation correspond à une position géographique précise (latitude, longitude, date), généralement obtenue via un service externe de géolocalisation. Une attraction représente un point d’intérêt touristique caractérisé par un nom et des coordonnées géographiques. Les attractions servent de référence pour calculer la proximité par rapport à la position de l’utilisateur.

Une récompense (UserReward) correspond aux points attribués à un utilisateur lorsqu’il visite une attraction. Ces points sont calculés à l’aide d’un service externe et sont cumulés afin d’influencer les offres de voyage proposées ultérieurement. Le terme proximité désigne la distance géographique calculée entre la position d’un utilisateur et une attraction, utilisée pour déterminer l’éligibilité à une récompense ou pour classer les attractions les plus proches.

Le Tracker est un composant d’arrière-plan chargé de mettre à jour périodiquement la position des utilisateurs et de déclencher le recalcul des récompenses. Il fonctionne de manière autonome et continue tant que l’application est en cours d’exécution.

Le terme endpoint REST désigne un point d’accès HTTP exposé par l’application permettant à un client externe de consommer les fonctionnalités (récupération de position, attractions proches, récompenses, offres). Un DTO (Data Transfer Object) est un objet simplifié utilisé pour structurer et transmettre les données vers le client sans exposer directement les modèles internes.

Enfin, la concurrence fait référence à la capacité de l’application à traiter plusieurs utilisateurs et tâches en parallèle. Elle s’appuie sur des mécanismes tels que CompletableFuture, des pools de threads (ExecutorService) et des collections thread-safe, afin de garantir à la fois la performance et la cohérence des données dans un contexte de forte charge.

3.3 Solutions techniques

Pour répondre aux attentes de performance et de fiabilité liées à la forte croissance du nombre d’utilisateurs, plusieurs solutions techniques ont été mises en place autour de la concurrence, de l’asynchronisme et de la structuration du code.

La première amélioration majeure concerne l’utilisation de traitements asynchrones grâce à CompletableFuture. Les opérations coûteuses, en particulier la récupération de la position GPS des utilisateurs et le calcul des récompenses, ne sont plus exécutées de manière bloquante. Elles sont désormais lancées en parallèle, ce qui permet d’exploiter efficacement les ressources CPU et de réduire drastiquement le temps global de traitement lorsque le volume d’utilisateurs augmente.

Afin de maîtriser cette exécution concurrente, des pools de threads dédiés (ExecutorService) ont été introduits dans les services principaux. Cette approche offre un contrôle explicite sur le nombre de threads utilisés et évite les dérives potentielles liées à une création non maîtrisée de threads. Elle garantit également une meilleure stabilité de l’application sous forte charge, notamment lors des tests de performance à grande échelle.

La gestion des données en environnement concurrent a été sécurisée par l’utilisation de structures de données thread-safe. Les utilisateurs sont stockés dans une ConcurrentHashMap, tandis que les listes de positions visitées et de récompenses utilisent des

CopyOnWriteArrayList. En complément, une synchronisation ciblée a été ajoutée lors de l’attribution des récompenses afin d’éviter les doublons et de garantir l’idempotence des opérations critiques.

Sur le plan des performances, une optimisation importante a consisté à charger les attractions en mémoire au démarrage de l’application. Cette décision permet d’éviter des appels répétés au service externe de géolocalisation pour récupérer la liste des attractions, réduisant ainsi la latence et la charge globale du système. Les calculs de distance et de proximité s’appuient ensuite sur ces données en mémoire.

La fonctionnalité de recommandation des attractions a été corrigée pour respecter la spécification attendue : l’application retourne systématiquement les cinq attractions les plus proches de la dernière position connue de l’utilisateur, indépendamment de leur distance. Les résultats sont triés par distance croissante et exposés via un DTO dédié, ce qui améliore la lisibilité et la consommation des données côté client.

Enfin, un pipeline d’intégration continue est présent (GitHub Actions, fichier `.github/workflows/maven.yml`) pour automatiser l’installation des dépendances locales, la compilation, les tests unitaires et la génération du JAR. Par défaut, ce pipeline exclut `TestPerformance` pour conserver des durées d’exécution raisonnables en CI, tandis que les tests de charge restent exécutables séparément pour valider les objectifs de performance.

3.4 Autres solutions non retenues

Plusieurs approches alternatives auraient pu être envisagées pour répondre aux problèmes de performance et de concurrence, mais elles n’ont pas été retenues afin de rester dans un cadre cohérent avec le périmètre du projet et l’existant.
Une première option aurait été de basculer l’application vers un modèle réactif (par exemple Spring WebFlux / Reactor) afin d’améliorer la gestion des appels I/O et la scalabilité. Cette solution n’a pas été choisie car elle implique une refonte plus lourde de la stack (contrôleurs, services, modèle d’exécution, tests), et l’objectif du projet était d’améliorer rapidement les performances sans

changer de paradigme ni introduire une complexité trop importante.

Une autre option courante aurait été d’utiliser des parallélisations “faciles” comme parallelStream(). Elle peut fonctionner dans certains cas, mais elle offre peu de contrôle (pool commun, comportement parfois difficile à diagnostiquer, risque d’interactions indésirables). Ici, un ExecutorService dédié et des CompletableFuture permettent un contrôle plus clair des ressources et un comportement plus prévisible sous forte charge.

Concernant la gestion des données, il aurait été possible d’introduire une base de données (et potentiellement un cache type Redis) afin de persister les utilisateurs et améliorer certains accès. Cette option a été écartée car le projet repose sur un mode “internal users” en mémoire, et l’enjeu principal était la performance des appels à des services externes et la concurrence, pas la persistance. Ajouter une base aurait aussi déplacé le problème (latence DB, modèle de données, migrations) au lieu de traiter la cause principale demandée dans le scénario.

Enfin, une approche “infrastructure” (scalabilité horizontale, conteneurisation avancée, orchestrateur type Kubernetes, microservices) aurait pu améliorer la montée en charge dans un contexte réel. Elle n’a pas été retenue car elle dépasse largement le cadre d’un projet de refonte ciblée : l’objectif était de rendre l’application plus rapide et plus stable à code constant, en optimisant l’exécution et la concurrence, et non de transformer l’architecture globale de déploiement